

Bono 2 - Combinaciones

Diseñe un programa que reciba n y r , con $0 \leq r \leq n$, y calcule el número de formas de escoger r objetos entre n objetos distintos sin importar el orden:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

El programa debe permitir:

1. Calcular $\binom{n}{r}$ para cualquier entrada válida
2. Verificar automáticamente la identidad $\binom{n}{r} = \binom{n}{n-r}$
3. Generar la fila n del triángulo de Pascal
4. Imprimir ejemplos de uso

Extensión opcional: generar todo el triángulo de Pascal hasta la fila n .

Descripción matemática

Combinaciones

Las combinaciones son una técnica de conteo para averiguar todas las formas en las que se puede seleccionar k elementos de un conjunto de n elementos. La fórmula para lograr esto es:

$$nCk = \frac{n!}{k!(n-k)!}$$

Nótese que es igual a la fórmula de la k -permutación, pero con un $k!$ en el denominador. Esto equivale a tomar todas las combinaciones y dividirlo por las $k!$ posibles órdenes de la selección, para contar solo las selecciones únicas. En otras palabras:

$$nCk = \frac{P(n, k)}{k!}$$

Es necesario que $k \leq n$, porque si no, $n - k$ es negativo, y por ende $(n - k)!$ no existe.

En la combinación el orden de selección de los elementos nunca importa. La permutación puede considerarse un superset de las combinaciones donde el orden sí importa.

Otra notación para las combinaciones la llamada **notación binomial**:

$$nCk = \binom{n}{k}$$

Esta notación también muy común está asociada con el **teorema de Newton** o teorema del binomio, que usa estos coeficientes para determinar los coeficientes de la potencia de un binomio. No por nada se llaman **coeficientes binomiales**.

Triángulo de Pascal

El triángulo de Pascal es una estructura matemática para ordenar los valores de los coeficientes de un triángulo de números. Cada número es la suma de los dos números directamente encima de él. La fila n del triángulo de Pascal corresponde a los coeficientes binomiales $\binom{n}{k}$ para $k = 0, 1, \dots, n$.

				1				
			1		1			
		1		2		1		
	1		3		3		1	
	1	4		6		4		1
	1	5	10		10	5		1
	1	6	15	20		15	6	1
	1	7	21	35	35	21	7	1

Figura 1: Triángulo de Pascal hasta el nivel 8

								$\binom{0}{0}$
				$\binom{1}{0}$		$\binom{1}{1}$		
		$\binom{2}{0}$		$\binom{2}{1}$		$\binom{2}{2}$		
	$\binom{3}{0}$		$\binom{3}{1}$		$\binom{3}{2}$		$\binom{3}{3}$	
	$\binom{4}{0}$	$\binom{4}{1}$		$\binom{4}{2}$		$\binom{4}{3}$		$\binom{4}{4}$
	$\binom{5}{0}$	$\binom{5}{1}$	$\binom{5}{2}$		$\binom{5}{3}$	$\binom{5}{4}$		$\binom{5}{5}$
	$\binom{6}{0}$	$\binom{6}{1}$	$\binom{6}{2}$	$\binom{6}{3}$		$\binom{6}{4}$	$\binom{6}{5}$	$\binom{6}{6}$
	$\binom{7}{0}$	$\binom{7}{1}$	$\binom{7}{2}$	$\binom{7}{3}$	$\binom{7}{4}$	$\binom{7}{5}$	$\binom{7}{6}$	$\binom{7}{7}$

Figura 2: Triángulo de Pascal, en formato de coeficiente binomial

De aquí podemos derivar algunas propiedades:

1. Como el triángulo es simétrico, los lados a la misma distancia de los extremos son iguales en valor. Esto es la **identidad de Pascal**:

$$\binom{n}{k} = \binom{n}{n-k}$$

2. Los extremos de las filas son 1, entonces:

$$\binom{n}{0} = \binom{n}{n} = 1$$

3. Sumar dos elementos consecutivos en una fila obtiene el elemento de debajo. Esta es la **regla de Pascal**:

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$$

Implementación

Combinaciones

De forma sencilla, sabemos que tenemos a la mano la librería `math`, la cual ofrece el método `math.comb`:

```
import math as m

print(m.comb(5, 3)) # -> 10
```

Antes de proceder con el código de nuestra versión, por supuesto, vamos a averiguar como funcionan las combinaciones para desarrollar optimizaciones.

Usando la regla de Pascal sabemos que:

$$\binom{n}{k} + \binom{n}{k-1} = \binom{n+1}{k+1}$$

Usando esto podemos derivar una forma recursiva de la combinación. La puedes verificar visualmente usando el triángulo de Pascal:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Por supuesto, si es recursivo, necesitamos un caso base. Los casos base son:

$$\binom{n}{0} = 1, \quad \binom{n}{n} = 1$$

Implementado en Python nos queda así. Usamos anotaciones de tipo para mejor legibilidad:

```
def recursive_combination(n: int, k: int) -> int:
    """Calcula combinaciones de forma recursiva usando la regla de Pascal"""

    # Error de tipo si no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if n < 0 or k < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    # Caso base
    if k == 0 or k == n:
        return 1

    # Recursión
    return recursive_combination(n - 1, k) + recursive_combination(n - 1, k - 1)
```

Sin embargo nos pueden preocupar los casos grandes, porque esta función es recursiva y se presta para altos consumos de memoria, calculos duplicados y un riesgo de `RecursionError` en Python. Para esto, podemos usar la **memoización** para guardar los resultados de las combinaciones ya calculadas y evitar cálculos repetidos:

```

cached = {}
def memoized_combination(n: int, k: int) -> int:
    if (n, k) in cached:
        return cached[(n, k)]

    # Error de tipo si no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if n < 0 or k < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    # Caso base
    if k == 0 or k == n:
        cached[(n, k)] = 1
        return 1

    # Recursión
    result = memoized_combination(n - 1, k) + memoized_combination(n - 1, k - 1)
    cached[(n, k)] = result
    return result

```

Esto reducirá los cálculos enormemente, pero igual el consumo de memoria empeora.

Para una última versión de la combinatoria, volvamos a la fórmula original:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Desenrollando el factorial $n!$ hasta llegar a $(n-k)!$ desarrollamos:

$$\begin{aligned}
 \binom{n}{k} &= \frac{n!}{k!(n-k)!} \\
 \binom{n}{k} &= \frac{n(n-1)(n-2) \cdot \dots \cdot (n-k+1)(\cancel{n-k})!}{k!(\cancel{n-k})!} \\
 \binom{n}{k} &= \frac{n(n-1)(n-2) \cdot \dots \cdot (n-k+1)}{k!}
 \end{aligned}$$

Y ahora podemos emparejar cada término del factorial $k!$ con cada término del numerador para obtener dos secuencias:

$$\begin{aligned}
 \binom{n}{k} &= \frac{n}{k} \cdot \frac{n-1}{k-1} \cdot \frac{n-2}{k-2} \cdot \dots \cdot \frac{n-k+1}{1} \\
 \binom{n}{k} &= \frac{n}{1} \cdot \frac{n-1}{2} \cdot \frac{n-2}{3} \cdot \dots \cdot \frac{n-k+1}{k}
 \end{aligned}$$

Podemos generalizarlo con un producto para cada secuencia respectivamente:

$$\binom{n}{k} = \prod_{i=0}^{k-1} \frac{n-i}{k-i} = \prod_{i=1}^k \frac{n-i+1}{i}$$

Esta es la forma más eficiente de calcular la combinación, al reducirlo a un producto. Sin riesgo de RecursionError y con un consumo de memoria constante. Usaremos la primera versión:

```
def iterative_combination(n: int, k: int) -> int:
    """Calcula combinaciones de forma iterativa usando la fórmula desenrollada"""

    # Error de tipo si no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if n < 0 or k < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    # Optimizamos usando la propiedad C(n, k) = C(n, n - k)
    k = min(k, n - k)

    result: int = 1
    for i in range(k):
        result *= (n - i) // (k - i)
        print(result)

    return result
```

Nótese que incluimos una optimización:

```
# Optimizamos usando la propiedad C(n, k) = C(n, n - k)
k = min(k, n - k)
```

Como lo dice el comentario, es una aplicación de la **identidad de Pascal**:

$$\binom{n}{k} = \binom{n}{n-k}$$

Esto nos permite reducir el número de términos a calcular, usando el número más pequeño de los dos disponibles sin alterar el resultado. Si funciona, está es la prueba de la identidad.

Triángulo de Pascal

Mientras me encontraba escribiendo este artículo para el bono, encontré en internet esta función en Typst (aunque la he modificado un poco) para generar los triángulos de Pascal:

[Fuente](#)

```
#let pascal_triangle(n) = {
  set align(center)
  let row = ()
  for r in range(0, n) {
    // step the row
    for i in range(row.len() - 1, 0, step: -1) {
      row.at(i) = row.at(i) + row.at(i - 1)
    }
    row.push(1)
    // print the row
    grid(
      columns: row.len() * (32pt,),
      align: center,
      //stroke : 0.2pt,
      ..row.map(str)
    )
  }
}
```

Por lo que decidiré reutilizarla en Python para hacer el triángulo. Su funcionamiento se basa en la propiedad:

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$$

Con la cual sobrescribe una fila para armar la siguiente, luego le añade un uno al final para completarla, imprime y repite el proceso. De esta forma, cada fila se construye a partir de la anterior, y se imprime a medida que se va construyendo:

```
def pascal_triangle(n: int):
    """Imprime el triángulo de Pascal con n filas"""

    row = []
    for _ in range(n):
        # Crea la fila
        for i in range(len(row) - 1, 0, -1):
            row[i] = row[i] + row[i - 1]
        row.append(1)

        # Imprime la fila
        print("\t".join(str(x) for x in row))
```

Comentarios y extras

Referencias

- <https://es.khanacademy.org/math/precalculus/x9e81a4f98389efdf:prob-comb/x9e81a4f98389efdf:combinations/v/introduction-to-combinations>
- <https://es.khanacademy.org/math/precalculus/x9e81a4f98389efdf:prob-comb/x9e81a4f98389efdf:combinations/v/combinacion-formula>
- https://es.wikipedia.org/wiki/Tri%C3%A1ngulo_de_Pascal
- https://es.wikipedia.org/wiki/Coeficiente_binomial
- <https://noesis.uis.edu.co/server/api/core/bitstreams/d0c3f853-dc1b-4742-9038-6e6b91650b95/content>